

Document number: P0880R0
Project: Programming Language C++
Audience: SG6 Numerics, Library Evolution

Igor Klevanets <cerevra@yandex.ru>, <cerevra@yandex-team.ru>
Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2018-01-31

Numbers interaction

Green lines are notes for the **editor** or for the **SG6** that must not be treated as part of the wording.

I. Introduction and Motivation

SG6 currently works on multiple new classes that represent numbers (see [P0539R2](#) and [P0037R4](#) for examples). However, those new classes do not interact well with each other.

This paper attempts to solve the problem in a generic way on a library level and provide wording for interactions of different new numeric types.

A proof of concept implementation available at: <https://github.com/cerevra/int/tree/master/v3>.

II. Changelog

- Initial version (split of the [P0539R2](#) paper)

III. Design

Imagine that we have numeric classes A, B and C and wish to provide wording for all the interactions of those types. In that case we'll have to describe 126 functions ((A, B, C) x (*, /, -, +, %, &, |, ^, <, >, <=, >=, ==, !=) x (A, B, C)). Such description tends to be error prone and scales badly. Addition of one more numeric class will force us to describe 224 functions.

To avoid that issue the proposal:

- defines Arithmetic and Integral concepts that require `std::numeric_limits<Arithmetic>::is_specialized` and `std::numeric_limits<Integral>::is_integer` to be true
- converts different numeric types to a single type using `common_type_t`

using the above approach we can define all the operations for numeric classes like this:

```
template<typename Arithmetic1, typename Arithmetic2>
common_type_t<Arithmetic1, Arithmetic2>
constexpr operator*(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `common_type_t<Arithmetic1, Arithmetic2>(lhs) * common_type_t<Arithmetic1, Arithmetic2>(rhs)`.

Each paper that proposes new numeric type has to specialize `numeric_limits` and `common_type` to get the interactions.

IV. Proposed wording

26.2 Definitions

[numerics.defns]

[Note to editor: Add the following paragraph to the end of [numerics.defns] - **end note]**

Define CT as `common_type_t<A, B>`, where A and B are the types of the two function arguments.

26.3 Numeric type requirements

[numeric.requirements]

[Note to editor: Add the following paragraphs to the end of [numeric.requirements] - **end note]**

Functions that accept template parameters starting with `Arithmetic` shall not participate in overload resolution unless `std::numeric_limits<Arithmetic>::is_specialized` is true.

Functions that accept template parameters starting with `Integral` shall not participate in overload resolution unless `std::numeric_limits<Integral>::is_integer` is true.

26.4 Numeric types interoperability

[numeric.interop]

Following operators are defined for types T and U if T and U have a defined `common_type_t<T, U>` and satisfy the `Arithmetic` or `Integral` requirements from [numeric.requirements] *[Note: Implementations are encouraged to provide optimized specializations of the following operators - end note]:*

[Note to SG6: We may add to `Arithmetic` and `Integral` concepts additional requirement that `std::numeric_limits<Integral-or-Arithmetic>::is_interoprable` shall be true. In that case no user code will be broken even if user already added following operators. However this seems to be an overkill. - **end note]**

```
template<typename Arithmetic1, typename Arithmetic2>
    common_type_t<Arithmetic1, Arithmetic2>
    constexpr operator*(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: CT(lhs) * CT(rhs).

```
template<typename Arithmetic1, typename Arithmetic2>
    common_type_t<Arithmetic1, Arithmetic2>
    constexpr operator/(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: CT(lhs) / CT(rhs).

```
template<typename Arithmetic1, typename Arithmetic2>
    common_type_t<Arithmetic1, Arithmetic2>
    constexpr operator+(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: CT(lhs) + CT(rhs).

```
template<typename Arithmetic1, typename Arithmetic2>
    common_type_t<Arithmetic1, Arithmetic2>
    constexpr operator-(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: CT(lhs) - CT(rhs).

```
template<typename Integral1, typename Integral2>
common_type_t<Integral1, Integral2>
constexpr operator%(const Integral1& lhs, const Integral2& rhs);
```

Returns: CT(lhs) % CT(rhs).

```
template<typename Integral1, typename Integral2>
common_type_t<Integral1, Integral2>
constexpr operator&(const Integral1& lhs, const Integral2& rhs);
```

Returns: CT(lhs) & CT(rhs).

```
template<typename Integral1, typename Integral2>
common_type_t<Integral1, Integral2>
constexpr operator|(const Integral1& lhs, const Integral2& rhs);
```

Returns: CT(lhs) | CT(rhs).

```
template<typename Integral1, typename Integral2>
common_type_t<Integral1, Integral2>
constexpr operator^(const Integral1& lhs, const Integral2& rhs);
```

Returns: CT(lhs) ^ CT(rhs).

```
template<typename Integral1, typename Integral2>
common_type_t<Integral1, size_t>
constexpr operator<<(const Integral1& lhs, const Integral2& rhs);
```

Returns: common_type_t<Integral1, size_t>(lhs) << static_cast<size_t>(rhs).

```
template<typename Integral1, typename Integral2>
common_type_t<Integral1, size_t>
constexpr operator>>(const Integral1& lhs, const Integral2& rhs);
```

Returns: common_type_t<Integral1, size_t>(lhs) >> static_cast<size_t>(rhs).

```
template<typename Arithmetic1, typename Arithmetic2>
constexpr bool operator<(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: CT(lhs) < CT(rhs).

```
template<typename Arithmetic1, typename Arithmetic2>
constexpr bool operator>(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: CT(lhs) > CT(rhs).

```
template<typename Arithmetic1, typename Arithmetic2>
constexpr bool operator<=(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: CT(lhs) <= CT(rhs).

```
template<typename Arithmetic1, typename Arithmetic2>
constexpr bool operator>=(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: CT(lhs) >= CT(rhs).

```
template<typename Arithmetic1, typename Arithmetic2>  
constexpr bool operator==(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: CT(lhs) == CT(rhs).

```
template<typename Arithmetic1, typename Arithmetic2>  
constexpr bool operator!=(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: !(lhs == rhs).

V. Feature-testing macro

For the purposes of SG10 we recommend the feature-testing macro name `__cpp_lib_num_interop`.